

Hot & Cold Food Temperature Monitoring

How to build the food-safety temperature feature as an extension of Ops Arrow's existing temperature module — reusing the notification, attachment, and issue-lifecycle systems already in the codebase.

.NET backend + React Native

Extend, don't rebuild

UK food-safety aligned

Scope: Backend (Domain / Application / Api / Infrastructure) + Mobile temperature feature

Basis: Grounded in a full read of the existing temperature code & reusable infrastructure

Status: Plan for review — no code written yet; key decisions pending sign-off

Prepared: 3 July 2026

Contents

1	The core insight	03
2	What exists today (grounded gap analysis)	04
3	What we reuse — the leverage	05
4	Decisions to lock first	06
5	Temperature rules & verdict model	07
6	Phased implementation (T1–T7)	08
7	Risks & food-safety nuances	10
8	Sequencing, effort & next step	10

How to read this

This plan is written against the *actual* code, not a blank page. Every "reuse" below names a real class, interface, or component that already ships in the app. Status tags: **EXISTS** **PARTIAL** **MISSING** — showing how much is genuinely new versus an extension of something already working.

1 The core insight

The existing temperature feature is a **single-band "in-range / out-of-range" logger**. The new specification adds two concepts that genuinely don't exist yet — but almost none of it needs new *infrastructure*.

Today, a monitoring unit has one `Min / Max` pair; a reading gets a binary `IsOutOfRange` flag; an out-of-range reading forces a free-text `ActionTaken`; and a whole-day manager sign-off closes the day. That's the whole model. The new requirements introduce:

1. **A three-tier verdict per food category** — Pass / Warning / Fail, computed differently for Hot vs Cold food (not a binary in/out flag).
2. **A separate, stateful equipment-issue lifecycle** — Working → Temperature Warning → Not Working → Under Maintenance → Resolved — with structured corrective actions, dwell-timers (hot 2h, cold 4h), manager alerts, and a per-incident manager approval to close.

Why this is an extension, not a rebuild

The app already has: a notification system with in-app/push/email/WhatsApp channels; an attachment (photo) pipeline; a proven "staff-raise → manager-resolve" lifecycle (Compliance actions); a status-column entity pattern (Coin Pod alerts); a signed manager-approval trail (Refusals); and a background-sweep alert job (temperature missed-checks). We assemble the new feature from these, extending the existing temperature units / readings / schedules / reports scaffolding rather than starting fresh.

2 What exists today

Grounded in a full read of the backend (`TemperatureLogService` , entities, controller) and the mobile feature (`temperatureLogsApi.ts` + 8 screens). This is the honest starting line.

REQUIREMENT AREA	STATUS	WHAT EXISTS / WHAT'S MISSING
Hot / cold food category on units	MISSING	<code>EquipmentType</code> (Fridge/Freezer/.../HotFoodDisplay) is the <i>appliance</i> , not a food category, and drives no thresholds.
Pass / Warning / Fail thresholds & messages	MISSING	Only binary <code>IsOutOfRange</code> (temp < min or > max) + a delta string. No warn tier, no category presets, no configurable target vs legal bands.
Equipment working-status lifecycle	MISSING	Unit has only an <code>IsActive</code> bool. No status states, no transitions.
Corrective-action selector	PARTIAL	Free-text <code>ActionTaken</code> exists (required when out of range) — but no predefined list/codes and no link to an incident.
Time-control timers (hot 2h / cold 4h)	MISSING	No dwell-timer anywhere. The predictive service returns a <code>minutesToBreach</code> but only surfaces it as a toast.
Manual " mark unit not working "	MISSING	No such operation; nearest is the <code>Inactive</code> toggle on unit-edit.
Resolution workflow + final-temp rules	MISSING	No resolution entity or fields anywhere.
Manager approval to close an incident	PARTIAL	Only mechanism is the whole-day sign-off (owner/manager). No per-incident approval; the sign-off API isn't even wired into any screen yet.
Photos on checks / issues	MISSING	None in the temperature feature (but the app-wide attachment pipeline exists — see §3).
Reports & history	PARTIAL	Date-range reports + PDF Print/Email/Share exist, but group by unit and count in/out-of-range only — no category or pass/warn/fail dimension.
Scheduled checks / missed-check alerts	EXISTS	Schedules, tolerance windows, and a missed-check push background service already ship — reused as-is.

Net: a single-band logger with scheduled-slot timing, trend prediction, and a daily sign-off. Categories, tiered thresholds, the equipment lifecycle, dwell-timers, resolution and per-incident approval are all net-new — but built on existing rails.

3 What we reuse — the leverage

Each row names a real, shipping part of the codebase we extend or copy — this is why the feature is far smaller than it looks.

NEW REQUIREMENT	EXISTING SYSTEM WE COPY / EXTEND
Manager alerts (in-app / push / email / WhatsApp, priority)	<code>INotificationService.SendAsync</code> + the <code>NotificationType</code> enum + <code>IsPriority</code> flag (plan-gated). Copy the multi-channel owner/manager fan-out from <code>ComplianceCheckService.NotifyComplianceActionRaisedAsync</code> (email + WhatsApp + attachments).
Timer / escalation background job	Copy <code>TemperatureMissedAlertsBackgroundService</code> — the per-shop sweep → per-device-token push pattern is exactly what a dwell-timer escalation needs.
Photos on checks & issues	<code>IAttachmentStorageService</code> + <code>CloseAttachmentStorage</code> (base64 → blob store, ≤10 MB). Copy the <code>ShiftCloseAttachment</code> entity → <code>TemperatureReadingAttachment</code> ; copy the <code>CloseDayScreen</code> picker on mobile. Files go to blob storage, not base64-in-DB.
Equipment issue lifecycle + manager approval	Template = Compliance actions. <code>ComplianceCheckService.CloseActionAsync</code> is the canonical "guard the transition → stamp actor/timestamp/notes → notify" handler. Borrow the <i>entity shape</i> from <code>CoinBagAlert</code> (a real <code>Status</code> enum + per-transition <code>...On / ...ByUserId</code> pairs). Optional signed approval = Refusals' <code>ReviewSignatureImagePath</code> .
Corrective-action selector	Shared <code>Chip</code> component — single-select like Compliance's <code>resultOptions</code> , multi-select like Refusals' <code>refusalReasons</code> . The existing free-text <code>ActionTaken</code> becomes structured codes.
Verdict badges, category filters, reports	<code>StatusBadge</code> , <code>SegmentedControl</code> , <code>DateRangeQuickPicks</code> , <code>QueryStateGate</code> , and the existing temperature PDF builders (<code>buildTemperatureRangeReportHtml</code> , <code>buildTemperatureScheduleGridHtml</code>).
Notification tap → deep link	Add the new alert types to <code>notificationTapNavigation.ts</code> (the <code>TEMPERATURE_LOG_TYPES</code> set already routes temperature pushes to the log screen).

Backend house style (so new code matches)

Not CQRS — **thin controller** → **application** `*Service` → **EF entity**. Enums one-per-file in `Domain/Enums`; entities extend `AuditableEntity`. Role gating is *doubled*: attribute (`[RequireShopRole]` / `[RequireFeature]`) on the action **plus** a redundant in-service check throwing `AppException(ErrorCodes.UnauthorizedRole, 403)`. The new code follows this exactly.

4 Decisions to lock first

These are genuine product / food-safety forks that change the **schema** — worth settling before any code.

⚠ Decision 2 is the biggest correctness risk — please resolve before build

Your `EquipmentType` already includes **Freezer** and **CoolRoom** (~ -18 °C). The spec's "Cold food 0–8 °C" would flag *every existing freezer as a Fail*. We must add a **Frozen** category (or per-unit freezer bands) or the feature misfires on day one.

#	DECISION	RECOMMENDATION
1	How do category rules relate to the per-unit range?	Add <code>FoodCategory {HotFood, ColdFood}</code> to the unit. Make Warn/Fail boundaries category constants (the law); the unit's <code>Min / Max</code> becomes the configurable ideal/target (Pass) band , defaulting to the category preset. A central <code>TemperatureEvaluator</code> computes the verdict.
2	Where do freezers / frozen units fit?	Add a Frozen category (e.g. Pass ≤ -18, Warn -17.9...-12, Fail > -12 — TBC) or per-unit bands for freezers. Blocking decision.
3	"Too hot" (> 70 °C hot food)?	Spec is silent above 70. Decide: still Pass, or a light "check food quality" note. (Recommend: Pass, no alarm.)
4	Equipment issue = new entity or status on unit?	New <code>TemperatureEquipmentIssue</code> entity (keeps history) + a denormalised <code>CurrentWorkingStatus</code> on the unit for fast list display. <code>IsActive</code> stays as "retired".
5	How are the 2h / 4h timers modelled?	Computed, not stored ticking. Store <code>IssueStartedOn</code> ; mobile derives elapsed/remaining; the background job escalates near-limit / at-limit / unresolved.
6	Signed manager approval on resolve?	Optional — add Refusals' signature-image pattern if a physical signature is wanted on close-out.

5 Temperature rules & verdict model

A single service — `TemperatureEvaluator` — becomes the one source of truth for the verdict, and is **unit-tested at every boundary**. It takes `(category, temperature, unitConfig)` and returns a `TemperatureResult` + message.

HOT FOOD holding at 63 °C or above

TEMPERATURE	RESULT	MEANING & ACTION
63 – 70 °C	PASS	Within safe hot-holding range.
60 – 62.9 °C	WARNING	Below required hot-holding — immediate attention; start the 2-hour time control.
Below 60 °C	FAIL	Unsafe — urgent action; open an equipment issue; alert the manager.

COLD FOOD chilled at 8 °C or below (fridge ideal 0–5 °C)

TEMPERATURE	RESULT	MEANING & ACTION
0 – 5 °C	PASS	Within ideal chilled range.
5.1 – 8 °C	WARNING	Within legal limit but higher than ideal — check door, stock, airflow.
Above 8 °C	FAIL	Unsafe — urgent action; start the 4-hour time control; alert the manager.

Equipment working status (separate from the reading verdict)

`EquipmentWorkingStatus` : Working · Temperature Warning · Not Working / Out of Service · Under Maintenance · Resolved · Inactive. The spec is explicit that this is *independent* of the temperature result — e.g. a hot cabinet can read Warning yet stay Working if staff confirm it's operating. Modelled as the `TemperatureEquipmentIssue` status column.

Legal line vs business target

The configurable internal target (e.g. hot 65–68) may only *tighten* the Pass band — it must never move the **legal Warn/Fail lines** (63 / 8 °C). Ops Arrow guides; the responsible person decides (spec §3). Messages stay advisory — no auto-discard, no auto-sell.

6 Phased implementation

Backend-first is essential — the mobile UX depends on the new verdict and issue APIs. Feature-gating (T7) threads throughout.

T1 Backend model & evaluation — the foundation

- **New enums** (one file each): `FoodCategory`, `TemperatureResult {Pass, Warning, Fail}`, `EquipmentWorkingStatus`, `TemperatureCorrectiveActionCode` (hot & cold action lists from spec §17).
- `TemperatureMonitoringUnit`: add `FoodCategory`, `CurrentWorkingStatus`, optional custom warn/fail overrides.
- `TemperatureReading`: add `Result`, structured corrective-action codes, optional issue link; new `TemperatureReadingAttachment` entity (copy `ShiftCloseAttachment`).
- **New** `TemperatureEquipmentIssue` entity (CoinBagAlert shape + Compliance close-out): `Status`, `IssueStartedOn`, opened/updated/resolved actor+timestamp pairs, corrective actions, `FinalTemperatureCelsius`, manager approval (+ optional signature).
- `TemperatureEvaluator` service — the single verdict source, with boundary unit tests (62.9/63.0, 5.0/5.1, 8.0/8.1).
- **EF migrations** + non-destructive backfill (existing units → default category from `EquipmentType`; existing readings → derive `Result` from `IsOutOfRange`).

T2 Backend service & API

- Extend `RecordReadingAsync`: compute `Result`; **require a corrective action on Warning/Fail**; on Fail auto-open/attach a `TemperatureEquipmentIssue` + set the unit's `CurrentWorkingStatus`; capture photos.
- New service ops (each a guarded transition stamping actor/time/notes, per `CloseActionAsync`): `MarkUnitNotWorkingAsync` (manual, §15), `SetUnderMaintenanceAsync`, `ResolveIssueAsync` (enforces §20 final-temp rules + manager approval), `ListIssuesAsync`, `GetIssueAsync`.
- New endpoints on `TemperatureLogsController`: `POST /units/{id}/mark-not-working`, `GET /issues`, `GET /issues/{id}`, `POST /issues/{id}/status`, `POST /issues/{id}/resolve` — house-style doubled role gating (raise = operational, resolve = owner/manager).
- Add `SaveTemperatureAttachmentsAsync` to `CloseAttachmentStorage`.

T3 Alerts & timers (background)

- Add `NotificationType` values: `HotFoodFail`, `ColdFoodFail`, `EquipmentNotWorking`, `TempTimerNearLimit`, `TempTimerLimitReached`, `TempIssueEscalation`, `TempIssueResolved`.
- On Fail / mark-not-working → **immediate** multi-channel owner/manager alert (copy `NotifyComplianceActionRaisedAsync`; `IsPriority=true` for the Urgent/Critical tiers in §18).
- New `TemperatureIssueTimerBackgroundService` (copy the missed-alerts sweep): near-limit (High) → limit reached (Critical) → unresolved-too-long (Escalation).
- Add the new types to mobile `notificationTapNavigation.ts` for tap deep-linking.

T4 Mobile — capture & verdict UX

- **Reading entry** (extend `TemperatureLogScreen`): category-aware **Pass/Warning/Fail banner** with the exact \$27/\$28 messages; **corrective-action chip selector** (required on Warn/Fail, "Other" → notes); **photo capture** (copy `CloseDayScreen` picker); the Warning/Fail follow-up question flows (§11–14).
- **Unit setup** (`TemperatureUnits` / `UnitEdit`): food-category selection + target band + presets.
- **Unit list**: current working-status + last-result badges + open-issue indicator (§22).

T5 Mobile — equipment issue lifecycle

- **Manual "Mark unit not working"** flow (§15) — reachable from the unit and the check screen.
- **Issue screen** (§22) with a **live timer countdown** (§16, computed client-side against 2h/4h).
- **Manager review / resolve** screens (§20/§23) — copy the role-gated `ComplianceCheckScreens` pattern; final-temperature resolution rules; manager approval (+ optional signature via `LandscapeSignatureModal`).

T6 Reports & history

- Extend reports (§24/§25): hot/cold **category filter**, **Pass/Warn/Fail breakdown**, equipment-not-working history, corrective-action history, discarded-food records, open/resolved issues, manager-approval history. Reuse the existing PDF builders + `DateRangeQuickPicks` .

T7 Config, entitlements & tests

- Feature-gate the new sub-capabilities (e.g. `temperature_log.equipment_issues` , `temperature_log.corrective_actions`) alongside the existing `temperature_log.*` keys; alert channels are already plan-gated.
- Optional shop config: custom target bands, editable corrective-action list.
- Tests: `TemperatureEvaluator` boundaries, lifecycle transition guards, migration/backfill.

7 Risks & food-safety nuances

- **Freezer / frozen category** (Decision 2) — the biggest correctness risk; resolve before build or existing freezer units flag Fail constantly.
- **Boundary precision** — 62.9 vs 63.0, 5.0 vs 5.1: use decimals with inclusive/exclusive rules exactly as the evaluator's unit tests pin them down.
- **"Guide, don't decide"** — keep every message advisory (spec §3); never auto-discard or auto-sell food.
- **Legal vs target** — the configurable internal target must never loosen the legal Fail line; Warn/Fail stay category-fixed.
- **Migration / backfill** of existing units & readings must be non-destructive and reversible.
- **Alert fatigue** — reuse the existing per-session dedupe from the missed-alerts service so a stuck failing unit doesn't spam managers every sweep.
- **Notification preferences are device-local today** (AsyncStorage, not server-synced) — if per-type manager routing is wanted, that's an added server-side model, not free.

8 Sequencing, effort & next step

Order: T1 → T2 → T3 (backend — the bulk), then T4 → T5 → T6 (mobile), with T7 threaded throughout. Mobile cannot start in earnest until the T2 verdict/issue APIs exist.

Effort

This is a **large, multi-sprint feature** — comparable in size to the entire Coin Pod module already shipped (new entities + service + controller + background job + a full set of mobile screens + reports). It is *not* a quick add-on; but because it rides on existing notification, attachment, and lifecycle rails, it's materially smaller than building any of those from scratch.

Recommended next step

Two things before code:

1. **Lock the six decisions in §4** — especially the freezer/frozen category and legal-vs-target — because they set the schema.
2. Then have the **T1 backend schema drafted** first (exact enums, entity fields, and the migration list) as the initial reviewable deliverable, before any service or UI work.